

Capstone Report: Maze Solver

What each algorithm did

BFS works out the shortest number of steps between every pair of important points — the start, each coin, and the goal — treating every open cell as equally easy to cross. **DFS** then tries out different orders for visiting the coins, using the distances BFS already worked out, and gives up on an order early as soon as it's already worse than the best one found so far, so it doesn't waste time checking it fully. **Dijkstra** does the same job as BFS, but takes the maze's terrain into account, since some cells cost more to walk through than others. **A*** finds the exact same distances as Dijkstra, but gets there faster by using the Manhattan-distance heuristic — a quick estimate of the straight-line distance to the goal — to point the search in the right direction, instead of spreading out evenly in every direction.

A vs. Dijkstra: how many cells each one checked*

Test maze	Coins	Dijkstra expanded	A* expanded	Reduction
tiny_2coins	2	162	103	36.4%
weighted_4coins	2	181	110	39.2%
medium_5coins	5	1,140	598	47.5%
stress_8coins	9	4,809	1,834	61.9%

A* checked fewer cells than Dijkstra every single time, and the gap grows as the maze gets bigger and more crowded with walls — from 36% fewer cells on the small maze up to 62% fewer on the hardest one. That's because the heuristic gives A* a good sense of which direction to search in, so it can skip cells that clearly aren't worth checking. Both algorithms still landed on exactly the same distances every time, as expected.

Best coin order — medium_5coins.txt (5 coins)

Uniform-cost (BFS + DFS): $S \rightarrow c(3,0) \rightarrow c(7,0) \rightarrow c(0,7) \rightarrow c(3,9) \rightarrow c(7,9) \rightarrow G$ — total steps: **32**.

Weighted (Dijkstra + A*): $S \rightarrow c(3,0) \rightarrow c(7,0) \rightarrow c(0,7) \rightarrow c(3,9) \rightarrow c(7,9) \rightarrow G$ — total cost: **32**.

Key points: 7 (start, 5 coins, goal) → 21 pairs computed by BFS. Coin orders: $5! = 120$ possible → 26 examined by DFS before confirming the optimum.